

MANUAL OPERATIVO

IFA SCADA

Operación, observabilidad y recuperación segura
Edge Windows y plataforma central

Versión 1.0 | 10 de agosto de 2026

Control del documento

Este PDF fue generado desde la fuente Markdown canónica incluida en el repositorio.

Campo	Valor
Documento	Manual operativo IFA SCADA
Versión	1.0
Fecha	2026-08-10
Plataforma edge	Windows 10/11 o Windows Server, x64
Plataforma central	Docker Compose, paquete runtime
Audiencia	Soporte técnico y administración de sistemas
Fuente canónica	deploy/manuales/manual-operacion-ifascada.md

Este manual explica el funcionamiento general, los comandos básicos de observabilidad y la recuperación segura del sistema. Los ejemplos usan marcadores como <IP_CENTRAL>, <EDGE_ID> y <PUERTO_COM>; deben sustituirse antes de ejecutarlos.

Clasificación de impacto

Marca	Significado
[LECTURA]	Consulta estado y no debe interrumpir el servicio.
[INTERRUPCIÓN]	Reinicia o detiene temporalmente un componente.
[CONFIGURACIÓN]	Cambia estado persistente; requiere respaldo y verificación.
[DESTRUCTIVO]	Puede eliminar datos. No se usa como diagnóstico rutinario.

Regla principal: observar primero, identificar la capa que falla, ejecutar una prueba mínima, aplicar una sola recuperación y verificar el resultado.

Índice

- 1. Diagnóstico rápido de cinco minutos
- 2. Funcionamiento general
- 3. Inventario operativo
- 4. Operación del edge Windows
- 5. Observabilidad del central
- 6. Diagnóstico por capas
- 7. Recuperaciones y casos frecuentes
- 8. Recolección de evidencias y escalamiento
- 9. Glosario de comandos

Diagnóstico rápido de cinco minutos

Ejecutar esta sección antes de reinstalar, editar SQL o modificar la configuración firmada.

1. Identificar la máquina y la hora

Dónde: edge Windows. **Privilegios:** usuario. **Impacto:** [LECTURA].

```
hostname
Get-Date
Get-NetIPAddress -AddressFamily IPv4 |
    Where-Object IPAddress -notlike "169.254*" |
    Select-Object InterfaceAlias, IPAddress
```

La hora es importante porque central y edge registran normalmente timestamps UTC. Una diferencia horaria puede hacer que datos recientes parezcan antiguos.

2. Confirmar tarea, servicio y proceso

Dónde: edge Windows. **Privilegios:** usuario; algunos detalles requieren administrador. **Impacto:** [LECTURA].

```
Get-ScheduledTask -TaskName "ifascada-edge" -ErrorAction SilentlyContinue
Get-ScheduledTaskInfo -TaskName "ifascada-edge" -ErrorAction SilentlyContinue |
    Format-List LastRunTime, LastTaskResult, NextRunTime

Get-Service -Name "ifascada-edge" -ErrorAction SilentlyContinue
Get-Process -Name "edge-agent" -ErrorAction SilentlyContinue |
    Select-Object Id, StartTime, CPU, WorkingSet64
```

Interpretación:

- Tarea presente y proceso presente: continuar con logs.
- Servicio presente y proceso presente: el edge usa NSSM.
- Tarea o servicio presente sin proceso: revisar `edge.task.log` y `edge.err.log`.
- No existe ninguno: el runtime no está instalado o usa nombres no estándar.

3. Leer los últimos logs

Dónde: edge Windows. **Privilegios:** usuario con lectura de `ProgramData`. **Impacto:** [LECTURA].

```
Get-Content "C:\ProgramData\ifascada\edge\logs\edge.out.log" -Tail 50
Get-Content "C:\ProgramData\ifascada\edge\logs\edge.err.log" -Tail 50
Get-Content "C:\ProgramData\ifascada\edge\logs\edge.task.log" -Tail 50
```

Buscar rápidamente:

```
$logs = @(
    "C:\ProgramData\ifascada\edge\logs\edge.out.log",
    "C:\ProgramData\ifascada\edge\logs\edge.err.log"
)

Select-String -Path $logs `
    -Pattern "ERROR|WARN|failed|mismatch|connected|mqtt|serial|print" |
    Select-Object -Last 100
```

4. Probar central y MQTT

Dónde: edge Windows. **Privilegios:** usuario. **Impacto:** [LECTURA].

```
$centralIp = "192.0.2.10" # Reemplazar por la IP real
```

```
Test-NetConnection $centralIp -Port 8088
Test-NetConnection $centralIp -Port 51883

Invoke-WebRequest `
  -Uri "http://${centralIp}:8088/health/live" `
  -UseBasicParsing `
  -TimeoutSec 5
```

Resultado normal:

- `TcpTestSucceeded` : True en ambos puertos.
- `/health/live` responde HTTP 200.

Si 8088 falla pero el edge inicia desde caché, la adquisición local puede continuar. Si 51883 falla, las publicaciones deben acumularse en `mqtt_outbox.db` hasta recuperar MQTT.

5. Confirmar configuración firmada y COM

Dónde: edge Windows. **Privilegios:** usuario. **Impacto:** [LECTURA].

```
Get-Item `
  "C:\ProgramData\ifascada\edge\runtime_config.signed.json", `
  "C:\ProgramData\ifascada\edge\mqtt_outbox.db", `
  "C:\ProgramData\ifascada\edge\ticket_sequence.db" `
  -ErrorAction SilentlyContinue |
  Select-Object Name, Length, LastWriteTime

Get-PnpDevice -Class Ports -PresentOnly |
  Select-Object Status, FriendlyName, InstanceId

[System.IO.Ports.SerialPort]::GetPortNames()
```

No editar `runtime_config.signed.json`: cualquier cambio manual invalida su integridad.

6. Comprobar el central Docker

Dónde: host central, desde `deploy\central-1.0.0-clean-runtime`. **Privilegios:** operador Docker. **Impacto:** [LECTURA].

```
docker info
docker compose -f .\docker-compose.yml --profile central --profile webui ps
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Clasificación rápida

Evidencia	Capa probable
No existe edge-agent	Instalación, tarea o servicio
edge_id mismatch	Identidad y caché firmada
failed opening serial port	COM, driver, USB o proceso competidor
API 8088 inaccesible	Red o Central API
MQTT 51883 inaccesible	Red o Mosquitto
Edge ok, tag antiguo	Balanza, COM, parser o ausencia de mensajes
Página de prueba no imprime	Spooler, USB, driver o impresora física
Página de prueba imprime, RAW no	Cola/driver incompatible con ESC-POS
buffer ... is empty	Secuencia de pesajes o buffer de automatización
device.command=Applied sin papel	Windows aceptó el trabajo; verificar canal RAW y salida física

Funcionamiento general

Flujo de adquisición

```
Balanza → puerto COM → edge-agent → MQTT → central-server
      → PostgreSQL/Redis → API/SSE → frontend
```

1. La balanza transmite texto por RS232.
2. Windows presenta el adaptador como un puerto COM.
3. `edge-agent` abre el puerto de forma exclusiva.
4. El driver `SerialAscii` separa tramas y aplica el parser configurado.
5. El pipeline del tag extrae valor, signo y unidad.
6. El edge publica telemetría y estados por MQTT.
7. El central persiste histórico en PostgreSQL/TimescaleDB y mantiene estado de tiempo real.
8. Redis distribuye eventos de tiempo real entre procesos.
9. La API y SSE exponen datos al frontend.

Flujo de configuración

La base central es la fuente de verdad del catálogo. El edge realiza:

1. `POST /api/edge/config/check` para conocer si cambió la configuración.
2. `GET /api/edge/config/runtime` cuando necesita una versión nueva.
3. Verificación de `edge_id`, hash, firma, algoritmo y `key_id`.
4. Escritura de la última configuración válida en `runtime_config.signed.json`.
5. Arranque desde caché firmada cuando la Central API no está disponible.

La prioridad de arranque es:

```
configuración remota firmada → caché local firmada → bootstrap legacy
```

El bootstrap legacy solo se usa cuando no se configuró `EDGE_CONFIG_URL`.

Flujo de resiliencia MQTT

Cuando MQTT no está disponible, el edge mantiene mensajes pendientes en SQLite:

```
C:\ProgramData\ifascada\edge\mqtt_outbox.db
```

La existencia o crecimiento del archivo no demuestra por sí solo una falla; debe correlacionarse con errores MQTT y la profundidad reportada en el heartbeat.

Flujo de impresión

```
Automatización → device.command → bytes ESC-POS → recurso compartido RAW
      → spooler Windows → puerto USB → impresora
```

El sistema usa `copy /B` hacia una ruta UNC. La impresora debe estar compartida y la cola usada por SCADA debe aceptar trabajos RAW. Una cola `Generic / Text Only` separada suele ser más confiable que la cola GDI del fabricante para ESC-POS.

`print.persist` registra intención o resultado lógico en central. No es un sensor de papel: no confirma físicamente que el mecanismo imprimió.

Estados operativos

- **Edge:** depende del heartbeat reciente.
- **Connection:** refleja el enlace del driver, por ejemplo COM abierto.

- **Device:** puede derivarse de la conexión o usar política `on_demand`.
- **Tag:** depende de la edad de la última muestra y su calidad.

Un edge puede estar `ok` mientras un tag está `stale`: el proceso y MQTT funcionan, pero no llegan muestras nuevas de esa variable.

Inventario operativo

Puertos predeterminados del central

Puerto host	Componente	Uso	Prueba
8088	Central API	HTTP, SSE y configuración de edge	<code>Test-NetConnection <IP_CENTRAL> -Port 8088</code>
3001	Web UI	Interfaz web	navegador o <code>Invoke-WebRequest</code>
51883	Mosquitto	MQTT externo para edges	<code>Test-NetConnection <IP_CENTRAL> -Port 51883</code>
55432	TimescaleDB	PostgreSQL externo	<code>pgAdmin</code> o <code>Test-NetConnection</code>
56379	Redis	Diagnóstico administrativo	<code>redis-cli ping</code> dentro del contenedor
58080	pgAdmin	Administración web PostgreSQL	navegador

Los valores pueden cambiar mediante `.env`. Confirmarlos con `docker compose config` y `docker compose ps`.

Contenedores operativos

Nombre	Función
<code>ifascada-timescaledb</code>	Catálogo, estado actual, históricos y auditoría
<code>ifascada-redis</code>	Caché/eventos de tiempo real
<code>ifascada-mosquitto</code>	Broker MQTT
<code>ifascada-pgadmin</code>	Administración de PostgreSQL
<code>ifascada-central-server</code>	Central API, consumo MQTT y persistencia
<code>ifascada-web-ui</code>	Frontend web

Rutas del edge Windows

Ruta	Contenido	Tratamiento
C:\Program Files\ifascada\edge\edge-agent.exe	Binario	Reemplazar solo mediante instalación/actualización controlada
C:\ProgramData\ifascada\edge\edge.env	Endpoints e identidad	No compartir; puede contener secretos
C:\ProgramData\ifascada\edge\run-edge.ps1	Supervisor	Generado por instalador
C:\ProgramData\ifascada\edge\runtime_config.signed.json	Caché firmada	Solo lectura
C:\ProgramData\ifascada\edge\config_apply_receipt.json	Recibo de aplicación	Solo lectura
C:\ProgramData\ifascada\edge\mqtt_outbox.db	Cola MQTT durable	No editar con el proceso activo
C:\ProgramData\ifascada\edge\ticket_sequence.db	Secuencia local de tickets	No borrar ni copiar sobre otra instalación
C:\ProgramData\ifascada\edge\logs\edge.out.log	Log principal	Consultable
C:\ProgramData\ifascada\edge\logs\edge.err.log	Errores del runner/binario	Consultable
C:\ProgramData\ifascada\edge\logs\edge.task.log	Transcript del supervisor	Consultable

Nombres lógicos

- **site**: planta o ámbito MQTT; actualmente puede conservar un código legacy como `plant-a`.
- **line**: línea lógica, por ejemplo `LCC`.
- **area**: agrupación funcional, por ejemplo `CABINAS DE PESAJE`.
- **cell**: unidad operativa, por ejemplo una cabina.
- **edge_code**: identidad única del agente.
- **connection_code**: enlace de transporte; el puerto COM pertenece a metadatos y puede cambiar.
- **device_code**: equipo físico.
- **tag_code**: variable publicada; debe ser única dentro del alcance exigido por el catálogo.

Operación del edge Windows

Detectar el modo de instalación

[LECTURA]

```
$task = Get-ScheduledTask -TaskName "ifascada-edge" -ErrorAction SilentlyContinue
$service = Get-Service -Name "ifascada-edge" -ErrorAction SilentlyContinue

[pscustomobject]@{
    ScheduledTask = $null -ne $task
    NssmService   = $null -ne $service
    Process       = $null -ne (Get-Process edge-agent -ErrorAction SilentlyContinue)
}
```

El modo recomendado actual es tarea programada al inicio de Windows. NSSM es opcional.

Estado de la tarea programada

[LECTURA]

```
Get-ScheduledTask -TaskName "ifascada-edge" |
    Select-Object TaskName, State

Get-ScheduledTaskInfo -TaskName "ifascada-edge" |
    Format-List LastRunTime, LastTaskResult, NextRunTime, NumberOfMissedRuns

Get-ScheduledTask -TaskName "ifascada-edge" |
    Select-Object -ExpandProperty Principal |
    Select-Object UserId, LogonType, RunLevel
```

El usuario de ejecución es relevante para impresoras compartidas. **SYSTEM** es simple para adquisición local, pero puede carecer de credenciales de red.

Estado del servicio NSSM

[LECTURA]

```
Get-Service -Name "ifascada-edge" |
    Select-Object Name, Status, StartType
```

Reinicio seguro: tarea programada

Dónde: edge. **Privilegios:** administrador. **Impacto:** [INTERRUPCIÓN].

Opción rápida cuando el supervisor sigue activo:

```
Stop-Process -Name "edge-agent" -Force -ErrorAction SilentlyContinue
Start-Sleep -Seconds 10
Get-Process -Name "edge-agent" -ErrorAction SilentlyContinue
```

run-edge.ps1 reinicia el binario después de cinco segundos.

Reinicio completo del supervisor:

```
Stop-ScheduledTask -TaskName "ifascada-edge" -ErrorAction SilentlyContinue
Stop-Process -Name "edge-agent" -Force -ErrorAction SilentlyContinue

Get-CimInstance Win32_Process -Filter "Name = 'powershell.exe'" |
    Where-Object CommandLine -like "*run-edge.ps1*" |
    ForEach-Object {
        Stop-Process -Id $_.ProcessId -Force -ErrorAction SilentlyContinue
    }

Start-ScheduledTask -TaskName "ifascada-edge"
Start-Sleep -Seconds 5
Get-Process -Name "edge-agent" -ErrorAction SilentlyContinue
```

Reinicio seguro: servicio NSSM

Dónde: edge. **Privilegios:** administrador. **Impacto:** [INTERRUPCIÓN].

```
Restart-Service -Name "ifascada-edge" -Force
Get-Service -Name "ifascada-edge"
Get-Process -Name "edge-agent" -ErrorAction SilentlyContinue
```

Seguir logs en tiempo real

[LECTURA]

```
Get-Content "C:\ProgramData\ifascada\edge\logs\edge.out.log" -Wait
```


En otra consola:

```
Get-Content "C:\ProgramData\ifascada\edge\logs\edge.err.log" -Wait
```

Revisar variables sin revelar secretos

[LECTURA]

```
$allowed = @(
    "EDGE_SITE", "EDGE_AGENT", "MQTT_HOST", "MQTT_PORT",
    "MQTT_CLIENT_ID", "EDGE_CONFIG_URL", "EDGE_RUNTIME_CACHE_PATH",
    "MQTT_OUTBOX_PATH", "EDGE_CONFIG_APPLY_RECEIPT_PATH"
)

Get-Content "C:\ProgramData\ifascada\edge\edge.env" |
Where-Object { $_ -match "=" } |
ForEach-Object {
    $key, $value = $_ -split "=", 2
    if ($allowed -contains $key.Trim()) {
        [pscustomobject]@{ Key = $key.Trim(); Value = $value.Trim() }
    }
} |
Format-Table -AutoSize
```

No publicar el archivo completo: contiene valores de enrolamiento y firma.

Verificar identidad y caché firmada

[LECTURA]

```
$envLine = Select-String `
    -Path "C:\ProgramData\ifascada\edge\edge.env" `
    -Pattern "^EDGE_AGENT="

$cache = Get-Content `
    "C:\ProgramData\ifascada\edge\runtime_config.signed.json" `
    -Raw |
    ConvertFrom-Json

[pscustomobject]@{
    EnvironmentEdge = ($envLine.Line -split "=", 2)[1]
    SignedEdge      = $cache.edge_id
    ConfigHash      = $cache.config_hash
    IssuedAt        = $cache.issued_at
}
```

Ambos edge ID deben coincidir. Si no coinciden, no editar el JSON: corregir identidad/configuración central y obtener un sobre firmado nuevo.

Observar archivos SQLite sin abrirlos

[LECTURA]

```
Get-Item `
    "C:\ProgramData\ifascada\edge\mqtt_outbox.db", `
    "C:\ProgramData\ifascada\edge\ticket_sequence.db" `
    -ErrorAction SilentlyContinue |
    Select-Object Name, Length, LastWriteTime
```

Un `LastWriteTime` reciente en outbox durante fallas MQTT indica que el edge continúa persistiendo publicaciones.

Cambiar únicamente el endpoint central

Dónde: carpeta del paquete runtime en el edge. **Privilegios:** administrador. **Impacto:** [CONFIGURACIÓN].

```
$centralIp = "192.0.2.10" # Reemplazar por la IP real

powershell -ExecutionPolicy Bypass `
  -File .\scripts\update-edge-endpoints.ps1 `
  -CentralHost $centralIp `
  -MqttPort 51883 `
  -CentralApiPort 8088
```

Verificar después `edge.env`, proceso y logs. Este script no cambia el catálogo de conexiones ni el `edge_id`.

Observabilidad del central

Todos los comandos de esta sección se ejecutan desde:

```
deploy\central-1.0.0-clean-runtime
```

El archivo operativo es `docker-compose.yml`. `docker-compose.scada.yml` pertenece al entorno de desarrollo desde código y no debe mezclarse con el runtime.

Recursos del host

Dónde: host central Windows. **Privilegios:** usuario. **Impacto:** [LECTURA].

```
Get-CimInstance Win32_OperatingSystem |
  Select-Object Caption, LastBootUpTime,
    @{N="RAMTotalGB";E={[math]::Round($_.TotalVisibleMemorySize/1MB,2)}},
    @{N="RAMLibreGB";E={[math]::Round($_.FreePhysicalMemory/1MB,2)}}

Get-CimInstance Win32_Processor |
  Select-Object Name, NumberOfCores, NumberOfLogicalProcessors, LoadPercentage

Get-Volume |
  Where-Object DriveLetter |
  Select-Object DriveLetter,
    @{N="SizeGB";E={[math]::Round($_.Size/1GB,2)}},
    @{N="FreeGB";E={[math]::Round($_.SizeRemaining/1GB,2)}}
```

Docker Engine y composición efectiva

[LECTURA]

```
docker version
docker info
docker compose version
docker compose -f .\docker-compose.yml config --services
```

No publicar `docker compose config` completo si resuelve secretos desde `.env`.

Estado de contenedores

[LECTURA]

```
docker compose -f .\docker-compose.yml --profile central --profile webui ps

docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Health checks detallados

[LECTURA]

```
$containers = @(
```

```

    "ifascada-timescaledb",
    "ifascada-redis",
    "ifascada-mosquitto",
    "ifascada-pgadmin",
    "ifascada-central-server",
    "ifascada-web-ui"
)

foreach ($name in $containers) {
    docker inspect $name `
        --format '{{.Name}} status={{.State.Status}} health={{if .
State.Health}}{{.State.Health.Status}}{{else}}n/a{{end}} restarts={{.RestartCount}}'
}

```

Logs de Central API y frontend

[LECTURA]

```

docker logs --tail 100 --timestamps ifascada-central-server
docker logs --tail 100 --timestamps ifascada-web-ui

```

Seguimiento:

```
docker logs --follow --since 5m ifascada-central-server
```

Filtrado:

```

docker logs --since 30m ifascada-central-server 2>&1 |
    Select-String "ERROR|WARN|mqtt|postgres|redis|config"

```

Endpoints HTTP

[LECTURA]

```

Invoke-WebRequest `
    -Uri "http://127.0.0.1:8088/health/live" `
    -UseBasicParsing `
    -TimeoutSec 5

Invoke-WebRequest `
    -Uri "http://127.0.0.1:3001" `
    -UseBasicParsing `
    -TimeoutSec 10

```

Consultas operativas:

```

Invoke-RestMethod "http://127.0.0.1:8088/api/edges/current"
Invoke-RestMethod "http://127.0.0.1:8088/api/connections/current"
Invoke-RestMethod "http://127.0.0.1:8088/api/devices/current"
Invoke-RestMethod "http://127.0.0.1:8088/api/tags/current"

```

En versiones con auditoría de impresión:

```
Invoke-RestMethod "http://127.0.0.1:8088/api/ops/prints?limit=20"
```

Si la API está protegida, usar el mecanismo de autenticación aprobado; no colocar credenciales en el historial de consola.

PostgreSQL/TimescaleDB

[LECTURA]

```
$dbUser = "USUARIO_DB" # Reemplazar
$dbName = "BASE_DB"    # Reemplazar

docker exec ifascada-timescaledb `
    pg_isready -U $dbUser -d $dbName -h 127.0.0.1 -p 5432
```

Tamaño de bases:

```
$dbUser = "USUARIO_DB" # Reemplazar
$dbName = "BASE_DB"    # Reemplazar

docker exec ifascada-timescaledb `
    psql -U $dbUser -d $dbName `
    -c "SELECT datname, pg_size_pretty(pg_database_size(datname)) FROM pg_database ORDER BY
pg_database_size(datname) DESC;"
```

Estado de edges:

```
SELECT
    e.edge_code,
    ecs.status,
    ecs.last_seen_at,
    EXTRACT(EPOCH FROM (NOW() - ecs.last_seen_at))::bigint AS age_seconds,
    ecs.outbox_depth,
    ecs.outbox_oldest_secs
FROM edges e
LEFT JOIN edge_current_state ecs ON ecs.edge_id = e.id
ORDER BY e.edge_code;
```

Estado de conexiones:

```
SELECT
    e.edge_code,
    c.connection_code,
    ccs.state,
    ccs.reason,
    ccs.last_seen_at
FROM connections c
JOIN edges e ON e.id = c.edge_id
LEFT JOIN connection_current_state ccs ON ccs.connection_id = c.id
ORDER BY e.edge_code, c.connection_code;
```

Edad de tags:

```
SELECT
    e.edge_code,
    t.tag_code,
    tcs.ts AS last_sample,
    EXTRACT(EPOCH FROM (NOW() - tcs.ts))::bigint AS sample_age_seconds,
    tcs.quality_json,
    tcs.value_json
FROM tags t
JOIN devices d ON d.id = t.device_id
JOIN edges e ON e.id = d.edge_id
LEFT JOIN tag_current_state tcs ON tcs.tag_id = t.id
ORDER BY sample_age_seconds DESC NULLS FIRST;
```

Las consultas de lectura pueden ejecutarse en pgAdmin. Cualquier **UPDATE** debe usar transacción, filtro específico, consulta previa y verificación posterior.

Redis

[LECTURA]

```
docker exec ifascada-redis redis-cli ping
docker exec ifascada-redis redis-cli info memory
docker exec ifascada-redis redis-cli info persistence
```

Resultado mínimo esperado: PONG.

Mosquitto

[LECTURA]

```
docker logs --tail 100 --timestamps ifascada-mosquitto
Test-NetConnection 127.0.0.1 -Port 51883
```

Suscripción de diagnóstico desde un host con `mosquitto_sub`:

```
$centralIp = "192.0.2.10" # Reemplazar por la IP real

mosquitto_sub `
  -h $centralIp `
  -p 51883 `
  -t "scada/+/edge/+/health/runtime" `
  -v
```

Para un edge concreto:

```
$centralIp = "192.0.2.10" # Reemplazar por la IP real
$site = "CODIGO_SITE"      # Reemplazar
$edgeId = "CODIGO_EDGE"    # Reemplazar

mosquitto_sub `
  -h $centralIp `
  -p 51883 `
  -t "scada/$site/edge/$edgeId/#" `
  -v
```

Usar usuario, contraseña y TLS cuando estén habilitados; no incluirlos en capturas o tickets.

Uso de disco Docker

[LECTURA]

```
docker system df
docker system df -v
docker volume ls
```

No ejecutar limpiezas automáticas antes de identificar qué imágenes, contenedores o volúmenes contienen datos requeridos.

Reiniciar un contenedor

Dónde: host central. **Privilegios:** operador Docker. **Impacto:** [INTERRUPCIÓN].

```
docker restart ifascada-central-server
docker logs --tail 50 --timestamps ifascada-central-server
Invoke-WebRequest "http://127.0.0.1:8088/health/live" -UseBasicParsing
```

Para frontend:

```
docker restart ifascada-web-ui
docker logs --tail 50 --timestamps ifascada-web-ui
```

No reiniciar TimescaleDB como primera respuesta a un fallo de UI.

Diagnóstico por capas

Capa 1: proceso edge

1. Confirmar tarea o servicio.
2. Confirmar edge-agent.
3. Leer `edge.task.log` si el proceso no aparece.
4. Leer `edge.err.log` para errores fatales.
5. Verificar tiempo de inicio del proceso.

Capa 2: identidad y configuración

1. Leer solo las claves no secretas de `edge.env`.
2. Comparar `EDGE_AGENT` con `edge_id` del sobre firmado.
3. Confirmar fecha y hash de la caché.
4. Buscar `config_sync_state` en heartbeat/logs.
5. Reiniciar el proceso solo después de que la fuente central sea correcta.

Capa 3: red

[LECTURA]

```
$centralHost = "central.empresa.local" # Reemplazar

Resolve-DnsName $centralHost -ErrorAction SilentlyContinue
Test-NetConnection $centralHost -Port 8088
Test-NetConnection $centralHost -Port 51883
route print
Get-NetAdapter | Select-Object Name, Status, LinkSpeed
```

Capa 4: COM/RS232

[LECTURA]

```
Get-PnpDevice -Class Ports |
    Select-Object Status, FriendlyName, InstanceId

Get-PnpDevice -Class Ports -PresentOnly |
    Select-Object Status, FriendlyName, InstanceId

[System.IO.Ports.SerialPort]::GetPortNames()
```

Para probar apertura exclusiva, primero detener el edge y cerrar VSPE/terminales.

[INTERRUPCIÓN]

```
Stop-ScheduledTask -TaskName "ifascada-edge" -ErrorAction SilentlyContinue
Stop-Process -Name "edge-agent" -Force -ErrorAction SilentlyContinue

$serialPortName = "COM3" # Reemplazar por el puerto que se diagnosticará
$serial = New-Object System.IO.Ports.SerialPort $serialPortName,9600,"None",8,"One"
try {
    $serial.Open()
    "PUERTO ABIERTO CORRECTAMENTE"
}
catch {
    "ERROR: $($_.Exception.Message)"
}
finally {
    if ($serial.IsOpen) { $serial.Close() }
```

```
$serial.Dispose()
}
```

Interpretación:

- **Acceso denegado:** otro proceso mantiene el puerto abierto.
- **Uno de los dispositivos conectados al sistema no funciona:** driver/USB bloqueado o adaptador averiado.
- **Apertura correcta:** el problema está en configuración del runtime, parámetros o parser.

Capa 5: MQTT

1. Probar TCP 51883.
2. Buscar `mqtt subscriptions ready`.
3. Buscar `mqtt publish ok` o errores de conexión.
4. Observar heartbeat con `mosquitto_sub`.
5. Revisar outbox y su edad.

Capa 6: Central API

1. Probar `/health/live`.
2. Consultar `/api/edges/current`.
3. Comparar estado de edge, conexión, dispositivo y tag.
4. Revisar logs de `ifascada-central-server`.
5. Verificar PostgreSQL y Redis antes de reiniciar central.

Capa 7: telemetría

Si el edge está `ok` pero no hay muestras:

1. Confirmar conexión `connected`.
2. Confirmar que la balanza transmite datos.
3. Revisar parámetros 9600/8/N/1 u otros configurados.
4. Revisar terminador de trama.
5. Revisar `regex/parser`.
6. Buscar mensajes de publicación del tag.
7. Comparar timestamp del último dato en PostgreSQL.

Capa 8: impresión

1. Confirmar que la automatización o API produce `device.command`.
2. Diferenciar `device.command` de `print.persist`.
3. Probar TCP 445 hacia el host de impresión.
4. Probar página Windows.
5. Probar RAW a la misma ruta UNC usada por SCADA.
6. Revisar spooler, cola, driver y puerto.

Prueba RAW:

```
$printerHost = "servidor-impresion" # Reemplazar
$rawShare = "IFA-SCADA-TMU220-RAW" # Reemplazar
$testFile = "$env:TEMP\ifascada-print-test.bin"
$bytes = @(27,64) + [System.Text.Encoding]::ASCII.GetBytes(
    "PRUEBA RAW IFA SCADA`r`n`r`n`r`n"
)

[System.IO.File]::WriteAllBytes($testFile,[byte[]]$bytes)
$printerUnc = "\\$printerHost\$rawShare"
& cmd.exe /C "copy /B `"$testFile`" `"$printerUnc`""
```

```
"EXIT CODE: $LASTEXITCODE"
Remove-Item $testFile -Force
```

Un exit code 0 confirma aceptación por Windows, no salida física. Verificar el papel.

Recuperaciones y casos frecuentes

Caso 1: CH340 visible, pero COM3 no abre

Síntoma: PnP muestra `USB-SERIAL CH340 (COM3)` como OK, pero `edge` y `SerialPort.Open()` devuelven “Uno de los dispositivos conectados al sistema no funciona”.

Evidencia: el error persiste con el `edge` detenido.

Prueba mínima: abrir COM3 con `.NET` después de cerrar todos los consumidores.

Recuperación:

1. Desconectar el adaptador USB.
2. Reiniciar Windows.
3. Conectar el adaptador.
4. Verificar el COM asignado.
5. Repetir la apertura directa.
6. Solo si persiste, reinstalar el driver WCH aprobado o probar un adaptador conocido como funcional.

Verificación: la prueba muestra `PUERTO ABIERTO CORRECTAMENTE` y el log contiene `serial-ascii connected on COM3`.

Caso 2: puerto COM ocupado

Síntoma: `Acceso denegado` al abrir el puerto.

Evidencia: el dispositivo aparece presente, pero solo falla mientras otro programa está abierto.

Prueba mínima: cerrar VSPE, terminales y herramientas del fabricante; detener `edge` y reintentar.

Recuperación: mantener un único propietario del puerto físico. VSPE solo puede coexistir si se configuró deliberadamente un divisor virtual.

Verificación: el `edge` abre el puerto y no reaparecen errores de acceso.

Caso 3: `edge_id mismatch`

Síntoma:

```
signed config edge_id mismatch: expected '<EDGE_NUEVO>' got '<EDGE_ANTERIOR>'
```

Evidencia: `EDGE_AGENT` no coincide con `edge_id` de la caché firmada.

Prueba mínima: comparar ambos valores sin modificar el JSON.

Recuperación: corregir el catálogo/identidad central y obtener una configuración firmada para el `edge` correcto. Retirar una caché antigua solo dentro de un procedimiento autorizado y con respaldo.

Verificación: el `edge` carga configuración remota/caché válida y arranca conexiones.

Caso 4: Central API no disponible

Síntoma: falla `EDGE_CONFIG_URL`, pero existe caché firmada válida.

Evidencia: el log indica arranque desde caché local.

Prueba mínima: comprobar 8088 y revisar `runtime_config.signed.json`.

Recuperación: restaurar red o central; no editar la caché. La adquisición puede continuar con la última configuración verificada.

Verificación: `/health/live` responde y el edge vuelve a reportar sincronización.

Caso 5: MQTT no disponible

Síntoma: errores de conexión/publicación y telemetría ausente en central.

Evidencia: 51883 falla y `mqtt_outbox.db` cambia.

Prueba mínima: `Test-NetConnection` y logs Mosquitto.

Recuperación: reparar broker/red. No borrar outbox.

Verificación: publicaciones vuelven, la profundidad del outbox disminuye y la telemetría aparece en central.

Caso 6: edge conectado, tag stale o disconnected

Síntoma: heartbeat reciente, conexión aparentemente activa, muestra antigua.

Evidencia: `sample_age_seconds` aumenta.

Prueba mínima: revisar tráfico COM y parser; comparar con un tag del mismo edge que sí se actualiza.

Recuperación: restablecer transmisión de balanza o corregir parámetros/metadatos en central. Reiniciar solo después del cambio.

Verificación: nuevo timestamp, calidad `Good` y edad reducida.

Caso 7: buffer vacío al imprimir

Síntoma: `device.command` falla con `buffer '<BUFFER_ID>' is empty`.

Evidencia: `print.persist` puede aparecer igualmente porque las acciones se auditan por separado.

Prueba mínima: confirmar que hubo valores positivos acumulados antes del disparo de impresión.

Recuperación: repetir el flujo correcto de pesaje o ajustar la automatización en la fuente central. No fabricar datos en SQLite.

Verificación: `device.command` queda `Applied`, contiene muestras y sale el ticket.

Caso 8: spooler Epson bloqueado

Síntoma: trabajos aceptados, cola o driver permanece en `Printing`, no sale papel.

Evidencia: impresora USB presente y cola normal, pero página/trabajos se atascan.

Prueba mínima: página de prueba Windows.

Recuperación:

Dónde: host conectado físicamente a la impresora. **Impacto:** [INTERRUPCIÓN].

```
$printerQueue = "NOMBRE_COLA" # Reemplazar

Restart-Service Spooler -Force
Get-Service Spooler
rundll32 printui.dll,PrintUIEntry /k /n $printerQueue
```

Verificación: sale la página y la cola vuelve a estado normal.

Caso 9: página Windows funciona, RAW no imprime

Síntoma: la página GDI sale, pero `copy /B` devuelve éxito sin papel.

Evidencia: API y `device.command` están `Applied`; el driver del fabricante descarta RAW.

Prueba mínima: enviar texto ESC-POS mínimo a la ruta UNC.

Recuperación: crear una cola separada sobre el mismo puerto físico con driver `Generic / Text Only`, compartirla con un nombre corto y configurar SCADA para usar esa ruta.

Ejemplo de creación, después de validar nombres:

```
$printerPort = "PUERTO_IMPRESORA" # Reemplazar por el puerto de la cola existente

Add-PrinterDriver -Name "Generic / Text Only"

Add-Printer `
  -Name "IFA SCADA TM-U220 RAW" `
  -DriverName "Generic / Text Only" `
  -PortName $printerPort `
  -Shared `
  -ShareName "IFA-SCADA-TMU220-RAW"
```

Verificación: la prueba RAW y la impresión desde API producen papel. Mantener la cola Epson original para impresión GDI.

Caso 10: recurso compartido inaccesible

Síntoma: `windows share print failed` o error de red/autenticación.

Evidencia: 445 falla, `Test-Path` no accede o la cuenta de la tarea no tiene permisos.

Prueba mínima:

```
$printerHost = "servidor-impresion" # Reemplazar

Test-NetConnection $printerHost -Port 445
Get-ScheduledTask -TaskName "ifascada-edge" |
  Select-Object -ExpandProperty Principal |
  Select-Object UserId, LogonType
```

Recuperación: usar una cuenta aprobada con acceso al share o corregir permisos del recurso. No habilitar acceso anónimo como solución rápida.

Verificación: `copy /B` funciona desde el contexto operativo y sale papel.

Caso 11: API registra Applied sin papel

Síntoma: `/api/ops/prints` muestra `device.command=Applied` y `print.persist=Applied`, pero no hay ticket.

Evidencia: el comando terminó sin error lógico; falta confirmar etapas Windows y físicas.

Prueba mínima: página Windows seguida de prueba RAW.

Recuperación: reparar spooler/USB si falla GDI; usar cola RAW si GDI funciona y RAW no.

Verificación: salida física y auditoría consistente.

Caso 12: central o frontend en error

Síntoma: UI no carga o devuelve error de servidor.

Evidencia: contenedor web activo, pero upstream central inaccesible; o central sin PostgreSQL/MQTT.

Prueba mínima: health central, logs de ambos y resolución interna.

Recuperación: corregir `CENTRAL_API_UPSTREAM`/host interno o restablecer dependencia concreta; recrear solo el servicio afectado cuando la configuración ya sea correcta.

Verificación: health HTTP 200, UI carga y APIs devuelven datos.

Acciones que no son diagnóstico ordinario

No ejecutar sin autorización, respaldo y objetivo explícito:

- `docker compose down -v;`
- `docker volume rm;`
- desinstalación con `-RemoveData;`
- `Remove-Item` recursivo sobre `C:\ProgramData\ifascada\edge;`

- DROP DATABASE o TRUNCATE;
- eliminación de mqtt_outbox.db o ticket_sequence.db;
- edición directa del sobre firmado.

Recolección de evidencias y escalamiento

Paquete mínimo de evidencia del edge

[LECTURA]

```
$stamp = Get-Date -Format "yyyyMMdd-HH:mm:ss"
$dest = Join-Path $env:TEMP "ifascada-edge-diagnostic-$stamp"
New-Item -ItemType Directory -Path $dest | Out-Null

Get-ComputerInfo |
    Select-Object WindowsProductName, WindowsVersion, OsBuildNumber |
    Out-File (Join-Path $dest "windows.txt")

Get-ScheduledTask -TaskName "ifascada-edge" -ErrorAction SilentlyContinue |
    Format-List * |
    Out-File (Join-Path $dest "task.txt")

Get-ScheduledTaskInfo -TaskName "ifascada-edge" -ErrorAction SilentlyContinue |
    Format-List * |
    Out-File (Join-Path $dest "task-info.txt")

Get-Process edge-agent -ErrorAction SilentlyContinue |
    Format-List Id,StartTime,CPU,WorkingSet64,Path |
    Out-File (Join-Path $dest "process.txt")

Get-PnpDevice -Class Ports |
    Format-Table Status,FriendlyName,InstanceId -AutoSize |
    Out-File (Join-Path $dest "ports.txt")

Copy-Item "C:\ProgramData\ifascada\edge\logs\*.log" $dest -ErrorAction SilentlyContinue

Get-Item "C:\ProgramData\ifascada\edge\runtime_config.signed.json",
    "C:\ProgramData\ifascada\edge\mqtt_outbox.db",
    "C:\ProgramData\ifascada\edge\ticket_sequence.db" `
    -ErrorAction SilentlyContinue |
    Select-Object Name,Length,LastWriteTime |
    Out-File (Join-Path $dest "data-files.txt")

$dest
```

No copiar `edge.env` al paquete sin sanitizarlo.

Evidencia del central

[LECTURA]

```
$stamp = Get-Date -Format "yyyyMMdd-HH:mm:ss"
$dest = Join-Path $env:TEMP "ifascada-central-diagnostic-$stamp"
New-Item -ItemType Directory -Path $dest | Out-Null

docker compose -f .\docker-compose.yml --profile central --profile webui ps |
    Out-File (Join-Path $dest "compose-ps.txt")

docker system df -v |
    Out-File (Join-Path $dest "docker-disk.txt")

docker logs --since 30m --timestamps ifascada-central-server 2>&1 |
    Out-File (Join-Path $dest "central.log")
```

```
docker logs --since 30m --timestamps ifascada-web-ui 2>&1 |
  Out-File (Join-Path $dest "web-ui.log")

docker logs --since 30m --timestamps ifascada-mosquitto 2>&1 |
  Out-File (Join-Path $dest "mosquitto.log")

$dest
```

Qué incluir al escalar

- Fecha, hora y zona horaria.
- Hostname, IP y `edge_id`.
- Síntoma observable y resultado esperado.
- Última vez conocida en que funcionó.
- Cambio reciente: reinicio, driver, cable, SQL, configuración o versión.
- Comando exacto ejecutado y salida completa.
- Primer error cronológico, no solo la última repetición.
- Resultado de la prueba discriminante.
- Recuperaciones ya intentadas, una por una.

Glosario de comandos

Sistema, procesos, tareas y servicios

Comando	Función	Dónde	Privilegios	Impacto
hostname	Identifica la máquina	Cualquiera	Usuario	Lectura
Get-Date	Confirma hora local	Cualquiera	Usuario	Lectura
Get-Process edge-agent	Comprueba proceso edge	Edge	Usuario	Lectura
Get-ScheduledTask -TaskName ifascada-edge	Comprueba tarea	Edge	Usuario	Lectura
Get-ScheduledTaskInfo -TaskName ifascada-edge	Última ejecución/resultado	Edge	Usuario	Lectura
Get-Service ifascada-edge	Comprueba servicio NSSM	Edge	Usuario	Lectura
Stop-Process -Name edge-agent -Force	Fuerza reinicio por supervisor	Edge	Administrador	Interrupción
Restart-Service ifascada-edge	Reinicia servicio NSSM	Edge	Administrador	Interrupción
Start-ScheduledTask -TaskName ifascada-edge	Inicia tarea	Edge	Administrador	Interrupción

Red, DNS y HTTP

Comando	Función	Dónde	Privilegios	Impacto
ipconfig	Muestra IP y gateway	Cualquiera	Usuario	Lectura
Resolve-DnsName <HOST>	Resuelve DNS	Cualquiera	Usuario	Lectura
Test-NetConnection <HOST> -Port <PUERTO>	Prueba TCP	Cualquiera	Usuario	Lectura
Invoke-WebRequest <URL> -UseBasicParsing	Prueba HTTP	Cualquiera	Usuario	Lectura
Get-NetAdapter	Estado de interfaces	Cualquiera	Usuario	Lectura
route print	Tabla de rutas	Cualquiera	Usuario	Lectura

Archivos y logs

Comando	Función	Dónde	Privilegios	Impacto
Get-Content <RUTA> -Tail 100	Últimas líneas	Edge/central	Usuario	Lectura
Get-Content <RUTA> -Wait	Sigue un log	Edge	Usuario	Lectura
Select-String -Path <RUTA> -Pattern <TEXT0>	Busca eventos	Edge/central	Usuario	Lectura
Get-Item <RUTA>	Tamaño y fecha	Cualquiera	Usuario	Lectura
Get-FileHash <RUTA> -Algorithm SHA256	Verifica integridad	Cualquiera	Usuario	Lectura

COM y PnP

Comando	Función	Dónde	Privilegios	Impacto
Get-PnpDevice -Class Ports -PresentOnly	Puertos presentes	Edge	Usuario	Lectura
[IO.Ports.SerialPort]::GetPortNames()	COM visibles para .NET	Edge	Usuario	Lectura
Disable-PnpDevice	Deshabilita adaptador	Edge	Administrador	Interrupción
Enable-PnpDevice	Habilita adaptador	Edge	Administrador	Interrupción
Get-CimInstance Win32_PnPSignedDriver	Versión del driver	Edge	Usuario	Lectura

Impresión

Comando	Función	Dónde	Privilegios	Impacto
Get-Printer	Lista colas y shares	Host impresora/edge	Usuario	Lectura
Get-PrintJob -PrinterName <COLA>	Trabajos de una cola	Host impresora	Usuario	Lectura
Get-Service Spooler	Estado de spooler	Host impresora	Usuario	Lectura
Restart-Service Spooler -Force	Reinicia cola Windows	Host impresora	Administrador	Interrupción
rundll32 printui.dll,PrintUIEntry /k /n <COLA>	Página de prueba	Host impresora	Usuario	Interrupción física
copy /B <ARCHIVO> \\<HOST>\<SHARE>	Envía RAW	Edge/host impresora	Usuario autorizado	Interrupción física
Add-Printer	Crea cola	Host impresora	Administrador	Configuración

Docker y central

Comando	Función	Dónde	Privilegios	Impacto
docker info	Estado Docker Engine	Central	Operador Docker	Lectura
docker compose ... ps	Estado del stack	Central	Operador Docker	Lectura
docker logs --tail 100 <CONTENEDOR>	Logs	Central	Operador Docker	Lectura
docker inspect <CONTENEDOR>	Estado y health	Central	Operador Docker	Lectura
docker stats --no-stream	CPU/RAM por contenedor	Central	Operador Docker	Lectura
docker system df -v	Uso de disco	Central	Operador Docker	Lectura
docker restart <CONTENEDOR>	Reinicia componente	Central	Operador Docker	Interrupción

PostgreSQL, Redis y MQTT

Comando	Función	Dónde	Privilegios	Impacto
pg_isready	Salud PostgreSQL	Central/contenedor	Usuario DB	Lectura
psql ... -c <SELECT>	Consulta SQL	Central	Usuario DB	Lectura si es SELECT
redis-cli ping	Salud Redis	Central/contenedor	Operador	Lectura
redis-cli info memory	Memoria Redis	Central/contenedor	Operador	Lectura
mosquitto_sub -t <TOPIC> -v	Observa MQTT	Host autorizado	Usuario MQTT	Lectura
Test-NetConnection <CENTRAL> -Port 51883	Prueba broker	Edge	Usuario	Lectura

Reinicios seguros

Objetivo	Comando principal	Verificación
Binario edge bajo supervisor	Stop-Process -Name edge-agent -Force	proceso reaparece y logs avanzan
Tarea edge completa	detener tarea/runner y Start-ScheduledTask	tarea Running y proceso presente
Servicio edge NSSM	Restart-Service ifascada-edge	servicio Running
Spooler	Restart-Service Spooler -Force	página de prueba
Central API	docker restart ifascada-central-server	/health/live HTTP 200
Frontend	docker restart ifascada-web-ui	puerto 3001 responde
Mosquitto	reiniciar solo tras revisar logs	puerto 51883 y heartbeat

Secuencia universal de diagnóstico

1. Registrar síntoma y hora. 2. Consultar estado sin cambiar nada. 3. Leer el primer error relevante. 4. Probar la frontera entre componentes. 5. Formular una sola causa probable. 6. Aplicar la recuperación mínima. 7. Repetir la prueba original. 8. Confirmar el flujo completo, no solo un estado intermedio.
